Documento de Arquitetura de Software

Dungeon Descend

Histórico de Revisão

Data	Versão	Descrição	Autor
15/04/2026	1.0	Elaboração inicial da documentação de arquitetura	Lucas Gabriel Pereira de Menezes e Equipe.
20/04/2026	1.1	Adicionado Fluxos de Processos, Premissas e restrições e os Diagramas	Lucas Gabriel Pereira de Menezes

1. Introdução

1.1. Finalidade

Este documento apresenta uma visão abrangente da arquitetura de software do projeto **Dungeon Descend**, um jogo de RPG Tático de Turnos (TRPG) com elementos de Roguelike. Ele detalha as decisões arquiteturais, tecnologias e padrões utilizados para garantir a escalabilidade e manutenção do sistema.

1.2. Escopo

O **Dungeon Descend** é um jogo digital focado em exploração de masmorras procedurais e combate estratégico em grid 2D isométrico. O escopo abrange desde a interface de usuário (menus e HUD) até a lógica de combate (IA de utilidade e gerenciamento de turnos), utilizando a Godot Engine como motor principal.

1.3. Definições, Acrônimos e Abreviações

Abreviação	Definição
TRPG	Tactical Role-Playing Game, um gênero de jogo focado em estratégia de combate por turnos.
GDD	Game Design Document, documento que detalha o design, mecânicas e conteúdo do jogo.
HUD	Heads-Up Display, a interface gráfica exibida na tela durante o jogo.

Godot	Motor de jogo de código aberto utilizado no projeto.
GDScript	Linguagem de programação de alto nível, baseada em Python, nativa da Godot Engine.
Singleton	Padrão de projeto de software que restringe a instanciação de uma classe a um único objeto. Na Godot, é implementado como Autoload.
IA de Utilidade	Um modelo de IA onde as decisões são tomadas com base na pontuação de "utilidade" de cada ação possível.
Cena/Nó	Estrutura fundamental da Godot Engine, onde Cenas são compostas por uma árvore de Nós.

1.4. Visão Geral

Este documento descreve a representação da arquitetura baseada na Godot Engine, as metas de desempenho, a visão lógica organizada em Singletons/Gerenciadores e a visão de implementação baseada em Cenas e Nós.

2. Representação da Arquitetura

A arquitetura do projeto é baseada no padrão de **Cenas e Nós (Scene/Node Tree)** nativo da Godot Engine. O sistema funciona como um cliente autônomo (Standalone Client), onde a lógica é desacoplada através de Singletons (Autoloads) que gerenciam o estado global do jogo.
O sistema opera como uma aplicação Cliente Standalone, sem dependências de servidores externos, alinhado ao escopo de um jogo single-player com modo cooperativo local.

2.1 Diagrama de Relações

O sistema organiza-se em três grandes blocos:

1. **Interface (UI/UX):** Gerenciada por Control Nodes (Menus, HUD).
2. **Mundo e Entidades:** Gerenciados por Node2D e TileMaps (Grid, Heróis, Inimigos).
3. **Core Managers (Singletons):** Scripts globais que coordenam as regras (TurnManager, GridManager, InputManager).

3. Metas e Restrições de Arquitetura

As decisões arquiteturais foram guiadas por um conjunto de metas e restrições técnicas para garantir a qualidade e a viabilidade do projeto.

Categoria	Meta ou Restrição	Justificativa

Desempenho	Manter uma taxa de 60 quadros por segundo (FPS) constantes.	Essencial para a fluidez da experiência do jogador em um jogo 2D.
Plataforma	Execução primária em Windows 10 e 11.	Foco no principal mercado de jogos para PC.
Motor Gráfico	Godot Engine 4.x	A Godot Engine 4 foi escolhida por ser um motor gráfico open source, gratuito e altamente otimizado para jogos 2D, oferecendo ferramentas nativas adequadas para pixel art e visão isométrica.
Linguagem	Utilização exclusiva de GDScript.	Maximiza a integração com a Godot Engine e simplifica o fluxo de desenvolvimento.
Persistência	Dados salvos localmente em formato JSON ou binário.	Evita a complexidade e o custo de uma infraestrutura de servidor.
Escopo	Exclusivamente single-player ou cooperativo local.	Restringe o escopo, evitando a complexidade do desenvolvimento de netcode para multiplayer online.
Estilo Visual	Gráficos 2D em Pixel Art com visão isométrica.	Define os requisitos para a camada de renderização e o formato dos assets.

4. Visão Lógica

4.1 Visão Geral: Pacotes e Camadas

A aplicação segue uma separação funcional baseada em responsabilidades:

4.1.1 Camada de Apresentação (Front-End)

Responsável pela renderização visual e interação com o usuário.

- **Control Nodes:** Implementam menus de Lobby, Loja e a HUD de combate contextual.
- **Node2D/TileMaps:** Gerenciam a malha tática (Grid), os biomas (Andares) e as sprites de personagens.

4.1.2 Camada de Lógica (Back-End)

Responsável pelas regras de negócio e processamento de dados.

- **Scripts em GDScript:** Lógica de jogo implementada diretamente nos Nós.

- **Singletons/Autoloads:** Gerenciadores globais que orquestram as principais funcionalidades do jogo.
    - **TurnManager:** Controla a Timeline de iniciativa e o fluxo de turnos.
    - **CombatResolver:** Processa cálculos de dano, rolagens de dados e modificadores elementais.
    - **GridManager**: Processa cálculo de coordenadas matemáticas do mapa e gerencia a interação com o grid.
    - **Utility AI:** Arquitetura de decisão para inimigos baseada em pontuação de utilidade (Scoring).
    - **InputManager:** Delega as entradas dos gamepads/teclado.

4.1.3 Camada de Dados (Persistência)

- **SaveManager:** Gerencia a leitura e escrita de progresso meta (Cristais) e configurações.
- **Custom Resources (.tres):** Arquivos de dados para definição de atributos de classes, armas e inimigos.

5. Visão de Implementação

5.1 Organização do Projeto

O diretório do projeto segue a estrutura padrão da Godot:

- **/scenes:** Contém todas as Cenas (**.tscn**) do jogo, que são os blocos de construção fundamentais. Exemplos: **Lobby.tscn**, **Combat.tscn**, **Hero.tscn**.
- **/scripts:** Contém todos os scripts GDScript (**.gd**). A estrutura de pastas espelha a estrutura das cenas para facilitar a navegação. Exemplo: **/scripts/entities/hero.gd**.
- **/assets:** Armazena todos os recursos de arte e áudio, como spritesheets (**.png**), trilhas sonoras (**.mp3, .ogg**) e fontes.
- **/autoloads:** Diretório dedicado aos scripts que serão configurados como Singletons globais, como **TurnManager.gd** e **SaveManager.gd**.
- **/resources:** Contém os Custom Resources (**.tres**) que armazenam dados de design, como **BarbarianClass.tres** ou **GoblinEnemy.tres**.

6. Fluxos de Processo

6.1 Ciclo de Jogo (Core Loop)

O ciclo de jogo principal é dividido em fases distintas.

1. **Lobby:** Formação do grupo (1 a 4 classes), ajuste de dificuldade (Medo), gasto de Cristais em Meta-Upgrades.
2. **Dungeon:** Entrada na sala, rolagem de iniciativa, combate tático por turnos, coleta de recompensa, escolha entre 2 portas, repetição até o Chefe do Andar.

3. **Transição de Andar:** Vencer Chefe do Andar, entrada na Sala de Descanso, avanço para o próximo Andar (Bioma/Região) com dificuldade aumentada.
4. **Fim:** Derrota de todo o grupo (Game Over) ou vitória sobre o Chefe Final do Jogo, retorno ao Lobby com Cristais ganhos para repetir o ciclo.

6.2 Fluxo de Combate Tático

O combate tático segue um fluxo detalhado [2]: 1.

1. **Início do Turno:** A Timeline aponta o turno do Herói do Jogador. A câmera centraliza no Herói e o Menu de Ações (HUD) é exibido. 2.
2. **Movimentação:** O jogador pode selecionar “Mover” ou mover o cursor para um tile válido. O sistema calcula e destaca tiles alcançáveis. Após a confirmação, o Herói se move e sua posição é atualizada.
3. **Seleção de Ação:** O jogador navega entre as abas (Action, Bonus Action, Items, End Turn) e seleciona uma ação. O sistema valida o custo da ação.
4. **Mira e Alvo:** O menu é minimizado, e tiles no grid que representam a Área de Alcance da arma/magia são destacados. A Linha de Visão (LoS) é validada. Um preview de combate (chance de acerto) pode ser exibido.
5. **Execução de Ataque:** O jogador confirma o alvo. O sistema calcula modificadores de dano, processa rolagem de dados, reproduz animações e efeitos sonoros. O dano é subtraído do HP do inimigo, e o resultado é exibido no HUD e no Log de Combate.
6. **Atualização de Estado:** O HP do inimigo, o estado da sala, a Timeline e os indicadores visuais de recursos são atualizados.
7. **Fim do Turno:** O jogador seleciona “End Turn” ou o sistema finaliza o turno automaticamente. A vez passa para o próximo ator na Timeline.

6.3 Gerenciamento de IA

A IA de combate utiliza uma arquitetura de Utilidade, onde cada inimigo avalia o campo de batalha e atribui notas numéricas a possíveis ações:

1. **Scanner de Cenário:** Coleta a posição de todos os heróis, seus status (HP/MP) e obstáculos sólidos no grid.
2. **Geração de Candidatos:** Cria uma lista de “Ações Possíveis” (ex: Mover + Atacar, Usar Poção + Magia, Recuar).
3. **Cálculo de Utilidade (Scoring):** Cada ação candidata recebe um score baseado no arquétipo do inimigo (Agressor, Caçador, Conjurador, Tático).
4. **Seleção e Execução:** A ação com o maior score é validada e enviada para o CombatResolver.

7. Visão de Dados

Esta visão detalha a estrutura de persistência dos dados do jogo.

7.1. Modelo de Dados de Persistência

O progresso permanente do jogador é salvo em um único arquivo (savegame.dat) no diretório de dados do usuário. Exemplo de estrutura de dados serializada:

```
{
  "meta_progress": {
    "crystals": 1200,
    "chaos_level_unlocked": 5,
    "permanent_upgrades": [
      {"id": "global_hp_boost_1", "purchased": true},
      {"id": "global_dmg_boost_1", "purchased": false}
    ]
  },
  "game_settings": {
    "master_volume": 0.8,
    "sfx_volume": 1.0
  }
}
```

- **meta_progress:** Agrupa todos os dados relacionados à progressão que persiste entre as partidas.
- **game_settings:** Armazena as configurações do jogador.

7.2. Modelo de Dados de Design (Custom Resources)

Os dados de design são gerenciados diretamente no editor da Godot através de **Custom Resources**, permitindo que os designers de jogo ajustem o balanceamento sem editar código. Cada recurso é um arquivo .tres que armazena os atributos de uma entidade ou item.
Exemplo (**BarbarianClass.tres**):

- **base_hp:** 150
- **base_strength:** 20
- **starting_weapon:** resource("res://resources/weapons/Axe.tres")

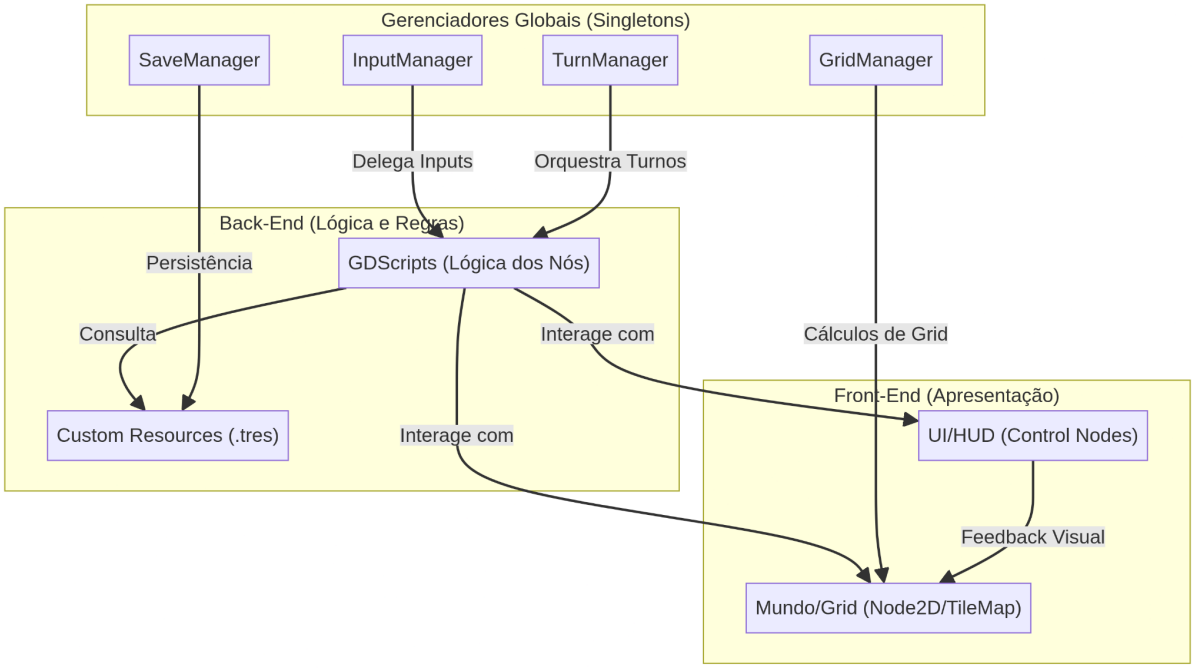
8. Premissas e Restrições

O projeto opera sob as seguintes premissas e restrições:

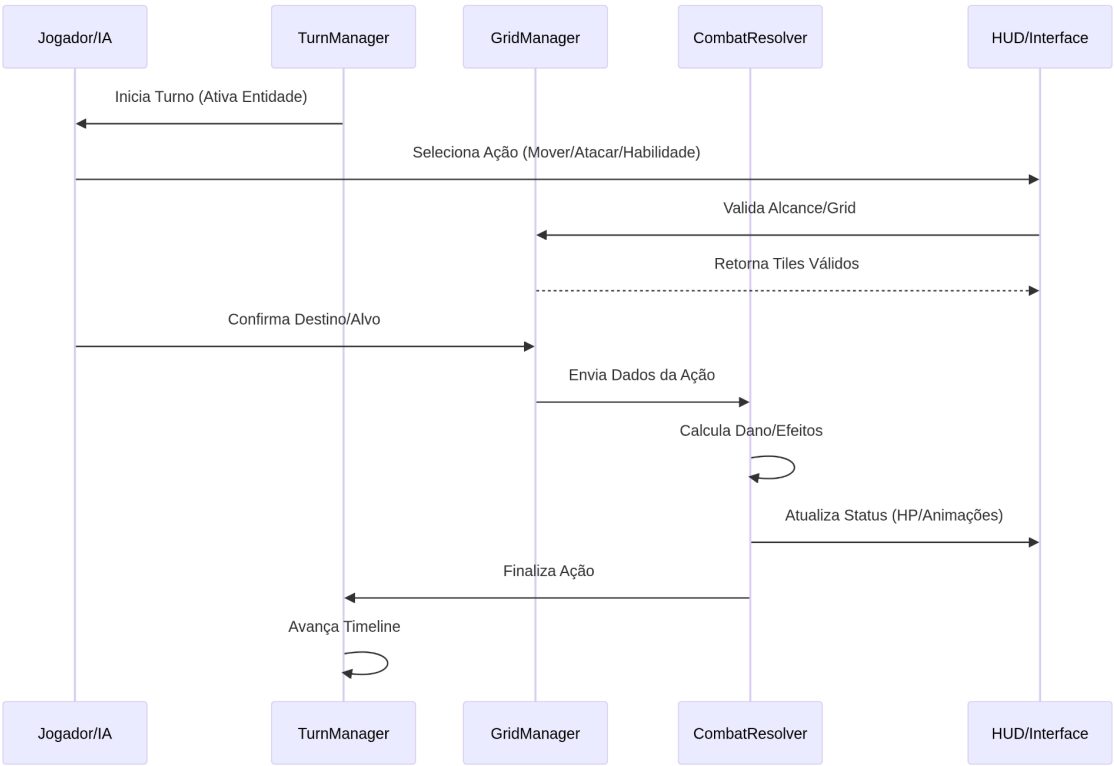
- **PR0001 - Ambiente 2D:** O desenvolvimento é estritamente restrito ao ambiente 2D top-down e isométrico da Godot, sem suporte a 3D.
- **PR0002 - Multiplayer Local:** O jogo é exclusivamente Single-Player ou Co-op Local. Não inclui infraestrutura de servidores de rede para multiplayer online.
- **PR0003 - Integração de Arte:** Recursos gráficos devem ser entregues como Sprite Sheets em .png sem compressão de filtro para garantir o visual “pixel perfect”.
- **PR0004 - Mudanças de Escopo:** Qualquer solicitação de mudança de escopo deve passar pelo Líder/PO e envolver reavaliação do cronograma.

9. Diagramas de Arquitetura

9.1 Diagrama de Componentes



9.2 Diagrama de Sequência: Fluxo de Combate



10. Referências

DOCUMENTO DE ARQUITETURA DE SOFTWARE. SysArq DOC. Disponível em: https://fga-eps-mds.github.io/2021-2-SysArq-Doc/doc_arquitetura/.
Godot Engine. Documentação Oficial. Disponível em: https://docs.godotengine.org/
Documento de Requisitos de Software - Dungeon Descend v1.2.
Game Design Document (GDD) - Dungeon Descend v0.0.6.
Especificação de Casos de Uso - Dungeon Descend v1.4.